Detailed Design

for

E-DBA

Version 1.4 approved

Prepared by Sibo Qiu 2230026131 Full
Yucheng Tong 2230026146 Full
Zheng Zhijinxiang 2230026222 Full
Manxi Tao 2230026145 Full
Yixin Xia 2230026171 Full
Boxin Su 2230026138 Full

A04

2025/4/9

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **ii**
1. Overview **1**
1.1 Project description 1
1.2 References 1
1.3 Design purpose 1
2. Overall description **2**
2.1 Class diagram 2
2.2 Refinements 2
3. Detailed design **4**
3.1 Class diagram 4
3.2 Classes 5
3.2.1 User 5
3.2.2 T-Admin 8
3.2.3 E-Admin 9
3.2.4 O-Convener 10
3.2.5 UserRegisManageSubsystem 10
3.2.6 ServiceConfiguration 11
3.2.7 CourseConfig 12
3.2.8 InterfaceConfig 13
3.2.9 Payment 14
3.2.10 DataUser 14
3.2.11 PrivateDataProvider 15
3.2.12 PrivateDataConsumer 15
3.2.13 PublicDataConsumer 16
3.2.14 Service 16
3.2.15 CourseService 17
3.2.16 ThesisService 17
3.2.17 StudentIdentityService 18
3.2.18 StudentRecordService 19
3.2.19 ActivityRecord (LOG) 19
3.2.20 UserQuestion 20
3.2.21 StudentRecordList 21
3.2.22 StudentRecord 21
3.2.23 Thesis 22
3.2.24 Email 22
3.2.25 DataSharingPolicy 23
3.2.26 BankAccount 24
3.2.27 Workspace 24
3.2.28 MemberList 24
4. Alternative detail design **25**
5. More considerations **25**

Revision History

Name	Date	Reason For Changes	Version

Manxi Tao	2025/4/9	Integrate documents	1.1
Manxi Tao	2025/4/9	Integrate documents	1.2
Yixin Xia	2025/4/9	Modify	1.3
Sibo Qiu	2025/4/9	Added Workspace, BankAccount, MemberList	1.4

1. Overview

1.1 Project description

More and more organizations need to share data with other organizations. Registering higher education institutions requires secure data sharing with other schools in a transparent and interoperable manner. This SRS specifies the software requirements for the Education Data Bay Area (E-DBA) system. The E-DBA system is designed to provide a secure and efficient platform for data sharing among registered higher education organizations. The scope of this SRS includes the functional and non-functional requirements of the entire system, covering features such as student identity authentication, thesis sharing, course information sharing, and an optional vault service. The SRS also outlines the system's integration with external payment gateways and interfaces for accessing public and private data.

1.2 References

[1] A04. (2025). SRS_a04_v2.6 (Version 2.6).
[2] A04. (2025). UI_diagram (Version 2.6).
[3] A04. (2025). Architecture Design (Version 1.0).

1.3 Design purpose

This modular architecture is designed to organize complex functionalities (payments, authentication, workspace management, etc.) into independent, cohesive modules, ensuring clear separation of concerns while maintaining well-defined interfaces for interaction. The primary goals are to enhance maintainability (enabling isolated updates without system-wide impact), improve scalability (supporting seamless addition of new features), and optimize development efficiency (allowing parallel work and reuse of common components). By isolating sensitive operations (e.g., payments) and standardizing reusable functions (e.g., email services), the design reduces technical debt, mitigates risks of cascading failures, and accommodates future business rule changes with minimal disruption. This approach is particularly suited for systems requiring frequent updates, strict security boundaries, or anticipated growth, balancing immediate development agility with long-term stability.

2. Overall description

2.1 Class diagram

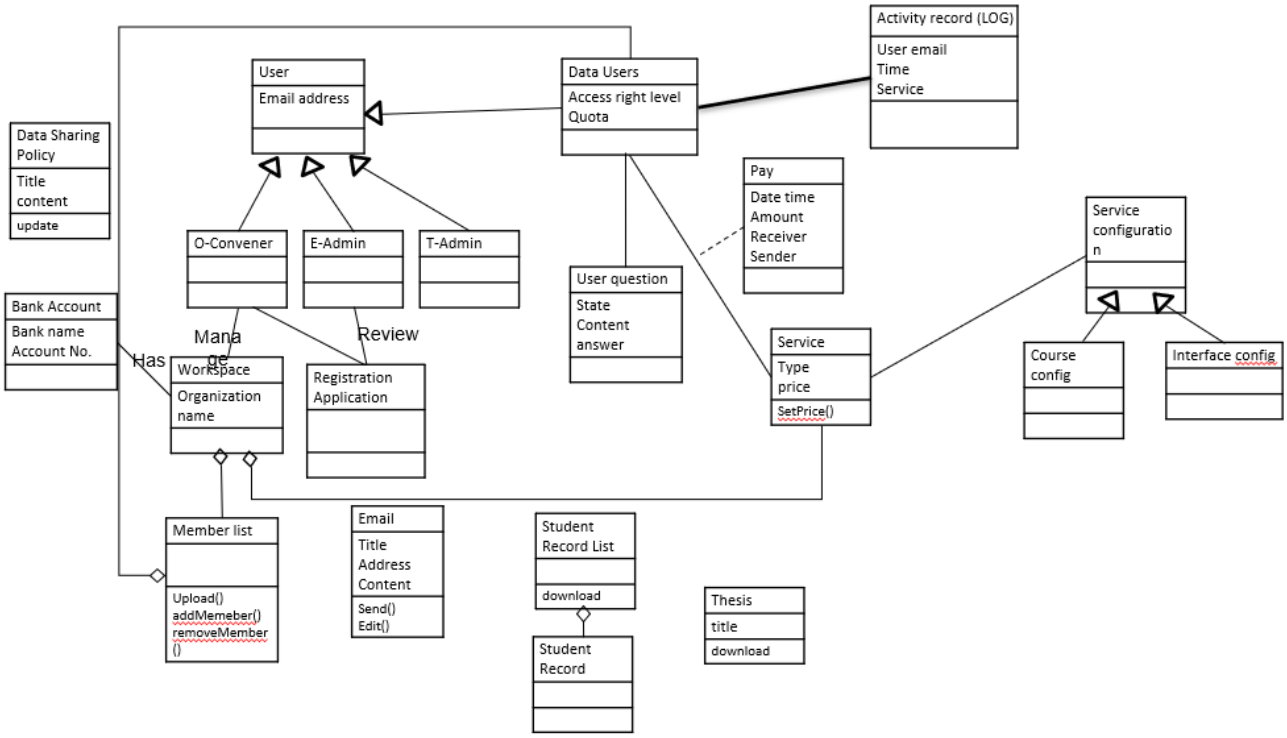


Figure 1 Original Class Diagram

2.2 Refinements

- **Functional Decomposition and Detailed Identification**

Each major function identified in the overall design has been decomposed into smaller, well-defined sub-functions. For example, rather than treating "user management" as a single block, the design now distinguishes between registration, authentication, access control, and help request handling.

Every sub-function is explicitly mapped to specific use cases. This mapping clarifies the expected behavior and interactions in scenarios such as user login, registration approval, and session management. These details directly influence how subsequent classes (e.g., User, Admin, and related help request functions) are implemented.

- **Detailed Class Structure and Method Refinement**

The class diagram has been refined by assigning each class a focused responsibility, give more details. For instance, the User class is now responsible for authentication and basic user operations, while the Admin and T-Admin classes handle more specific system management tasks. This segregation simplifies maintenance and future updates.

Access Modifiers: Methods and attributes now explicitly define their access levels (public, private, or protected), ensuring that internal implementation details remain encapsulated.

Method Contracts: Each method (such as login(), logout(), or setUserRole()) includes clearly stated preconditions (e.g., valid session or email format), postconditions (e.g., token invalidation, record updates), and specific error return codes (e.g., EMAIL_NOT_REGISTERED, INVALID_EMAIL_FORMAT). These method-level contracts are directly referenced in the design of later classes, ensuring consistency across the system.

- **Specification of Constraints and Exception Handling**

  Constraints have been rigorously defined for method to ensure robust operation. For example:

  Login Method: Preconditions now include that the email must be non-null, follow a valid format, and be registered in the system. Postconditions detail the process of generating and sending a verification code.

- **Optimization of Class Relationships and Structure**

  The relationships among classes (inheritance, association, composition) have been re-evaluated and adjusted to reduce dependencies. For example, responsibilities originally combined within a single class have been segregated into the User, Admin, and specialized service classes, promoting modularity and isolating changes.

  Detailed interface contracts have been defined for interactions between classes. This is evident in the communication protocols established between the UserRegisManageSubsystem and various administrative classes (e.g., T-Admin, E-Admin, and O-Convener).

  The class restructuring directly supports future scalability improvements.

3. Detailed design

3.1 Class diagram

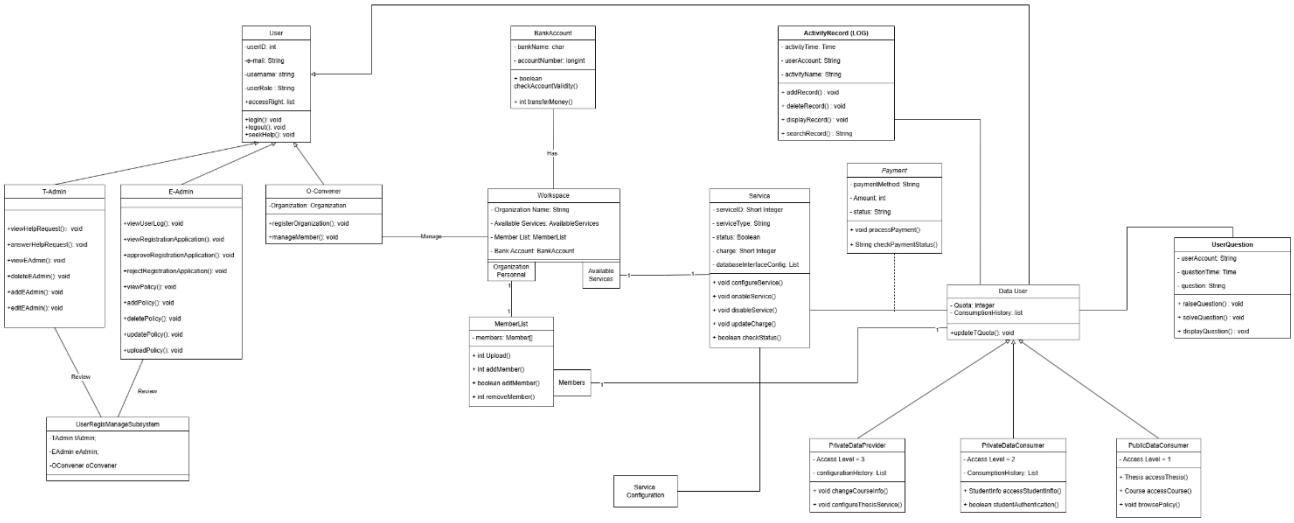


Figure 2 Class Diagram in Details I

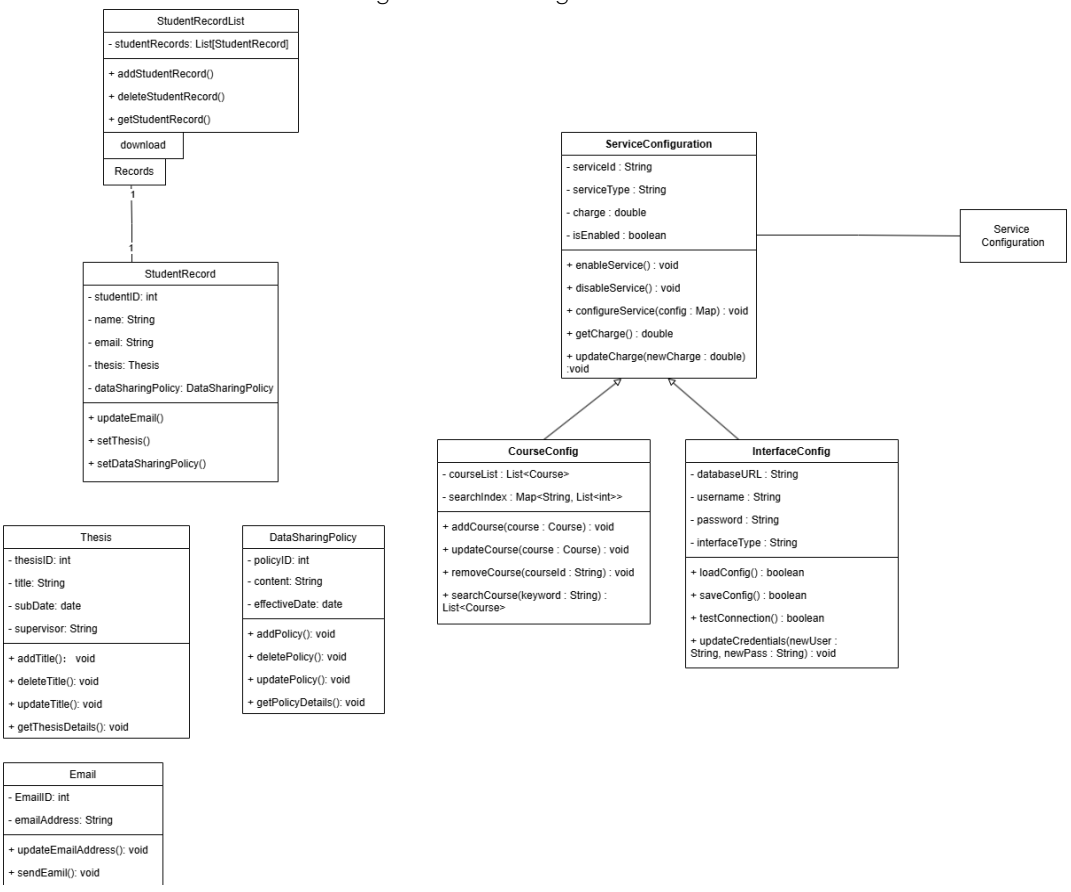


Figure 3 Class Diagram in Details II

3.2 Classes

3.2.1 User

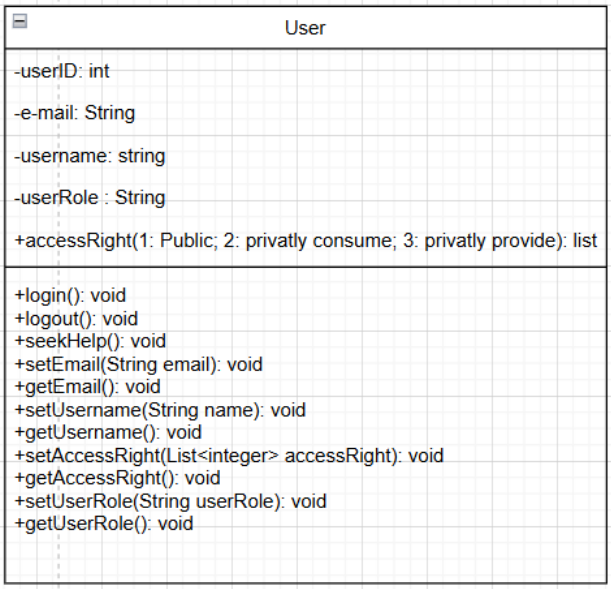


Figure 4 User

Explanations

- **userID**: Unique identifier for the user. And this is immutable and automatically generated by the system during registration.
- **email**: User's email address, used for login and system communication, which must adhere to standard email format (e.g., xxx@domain.com).
- **username**: User's display name customizable and used to identify the user within the system.
- **userRole**: Defines the user's role, which determines their permissions.
- **accessRight**: List of access rights, including three types. Rights must align with the userRole.
- **login():** Authenticates the user using email and password.
- **logout():** Terminates the user's active session and releases system resources.
- **seekHelp():**Allows the user to request assistance
- **setEmail()/getEmail():** Sets or retrieves the user's email address.
- **setUsername()/getUsername():** Sets or retrieves the user's display name.
- **setAccessRight()/getAccessRight():** Sets or retrieves the user's access rights list.
- **setUserRole()/getUserRole():** Sets or retrieves the user's role.

Constraints

login():

Precondition: Email Validation:
email must not be `null` or empty.
email must adhere to a valid format (e.g., `xxx@domain.com`).
email must be registered in the system (existing user).

Postcondition:

- **Failure Cases:**
  - If the email is not registered, return **EMAIL_NOT_REGISTERED**.
  - If the email format is invalid, return **INVALID_EMAIL_FORMAT**.
- **Success Case:**
  - If the email is valid and registered:
  - Generate a verification code and send the code to the user's email.
  - Return **VERIFICATION_CODE_SENT**.

logout():

Precondition: User must be logged in (an active session exists).

Postcondition:

- **Failure Case**:
  - If no active session exists, return **NO_ACTIVE_SESSION**.
- **Success Case**:
  - Terminate the active session.
  - Clear session-related data (e.g., tokens, cached permissions).
  - Return **LOGOUT_SUCCESS**.

seekHelp():

Precondition: User must be logged in (an active session exists).

Postcondition:

- **Failure Case**:
  - If no active session exists, return **SESSION_REQUIRED**.
- **Success Case**:
  - Open a support ticket with the user's session details (e.g., userID, timestamp).
  - Send a confirmation message to the user's registered email.
  - Return **HELP_REQUEST_SUBMITTED**

setEmail(String email):

Precondition:

- **email** must not be **null** or empty.
- **email** must adhere to a valid format (e.g., **xxx@domain.com**).
- Email must not be registered before.

Postcondition:

- **Failure Cases**:
  - If email format is invalid, return INVALID_EMAIL_FORMAT.
  - If the email is already registered by another user, return EMAIL_ALREADY_EXISTS.
- **Success Case**:
  - Update the user's email in the database.
  - Send a confirmation email to the new address.
  - Return **EMAIL_SET_SUCCESS**.

getEmail():

Precondition:

- User must be logged in.

Postcondition:

- Return the user's email as a **String**.
- If no email is associated with the account, return **NO_EMAIL_ASSOCIATED**.

setUsername(String name):

Precondition:

- **name** must not be **null** or empty.
- User must be logged in.

Postcondition:

- **Failure Cases**:
  - If the username is already taken, return **USERNAME_UNAVAILABLE**.
- **Success Case**:
  - Update the username in the database.
  - Return **USERNAME_ SET_SUCCESS**.

getUsername():

Precondition:

- User must be logged in.

Postcondition:

- Return the user's display name as a **String**.

setAccessRight(List<Integer> accessRight):

Precondition:

- **accessRight** list must only contain valid values (**1**, **2**, **3**).
- User must be logged in.
- User's role must allow permission modifications (e.g., Admin or authorized roles).

Postcondition:

- **Failure Cases**:
  - If the list contains invalid values, return **INVALID_ACCESS_RIGHT**.
  - If the user's role conflicts with the permissions return **ROLE_PERMISSION_CONFLICT**.
- **Success Case**:
  - Update the user's access rights.
  - Log the change with administrator ID (if applicable).
  - Return **ACCESS_RIGHT_UPDATED**.

getAccessRight():

Precondition:

- User must be logged in.

Postcondition:

- Return the user's access rights as a **List<Integer>**.

setUserRole(String userRole):

Precondition:

- **userRole** must be a valid role
- The current user must have administrative privileges (**Admin** role).

Postcondition:

- **Failure Cases**:
  - If the role is invalid, return **INVALID_USER_ROLE**.
  - If the current user is not an Admin, return **PERMISSION_DENIED**.
- **Success Case**:
  - Update the user's role.
  - Adjust permissions automatically (e.g., revoke **Admin**-only rights if role is downgraded).
  - Return **USER_ROLE_UPDATED**

getUserRole():

Precondition:

- User must be logged in.

Postcondition:

- Return the user's role as a **String**.

logout():

Precondition: User is logged in (currentSession != null).

Postcondition: Invalidates the session token and clears cached data.

setAccessRight(List<Integer> accessRight):

Precondition: Input list must only contain 1, 2, 3. Consumer roles cannot include 3 (privately provide).

Postcondition: Logs the administrator ID and timestamp of the change.

setUserRole(String userRole):

Precondition: Only users with Admin role can invoke this method.

Postcondition: Role downgrades (e.g., Admin → Consumer) revoke admin-exclusive rights.

seekHelp():

Precondition: User is logged in with a valid session (sessionTimeout > currentTime).

Postcondition: Saves the request to a support log system and notifies the support team.

3.2.2 T-Admin

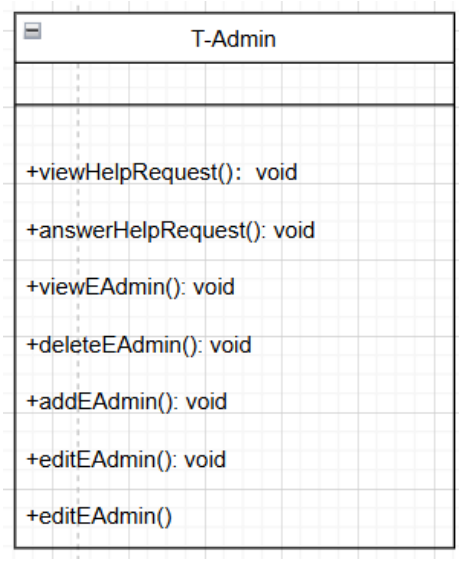


Figure 5 T-Admin

Explanations

- **viewHelpRequest():** Displays a list of all user-submitted help requests, including details like request ID, timestamp, and status (e.g., pending/resolved).
- **answerHelpRequest()**: Allows administrators to respond to a specific help request
- **viewEAdmin()**: Retrieves and displays information about all registered E-Admins.
- **deleteEAdmin()**: Removes an E-Admin account from the system. Requires confirmation and logs the action for auditing.
- **addEAdmin()**: Creates a new E-Admin account by specifying required fields.
- **editEAdmin()**: Modifies an existing E-Admin's properties

3.2.3 E-Admin

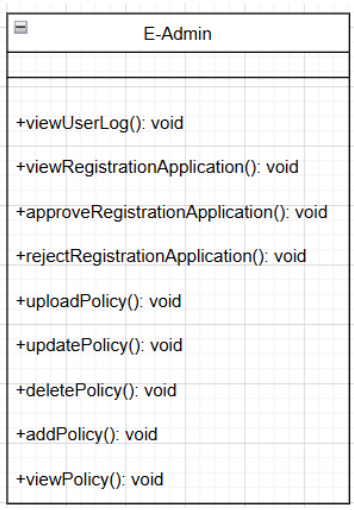


Figure 6 E-Admin

Explanations

- **viewUserLog():** Shows a detailed log of user activities (e.g., login attempts, password changes, access requests).
- **viewRegistrationApplication()**: Lists pending user registration applications with details (e.g., applicant email, submission date).
- **approveRegistrationApplication()**: Approves a registration request, granting the user full access to the system. Triggers a confirmation email.
- **rejectRegistrationApplication():** Rejects a registration request, optionally providing a reason to the applicant via email.
- **viewPolicy():** Displays the current system policies and their versions.
- **addPolicy():** Adds a new policy to the system.
- **deletePolicy():** Removes an outdated or inactive policy from the system. Requires administrative privileges.
- **updatePolicy():** Edits an existing policy's content or metadata.
- **uploadPolicy():** Uploads a policy document to the system, linking it to a policy entry.

3.2.4 O-Convener

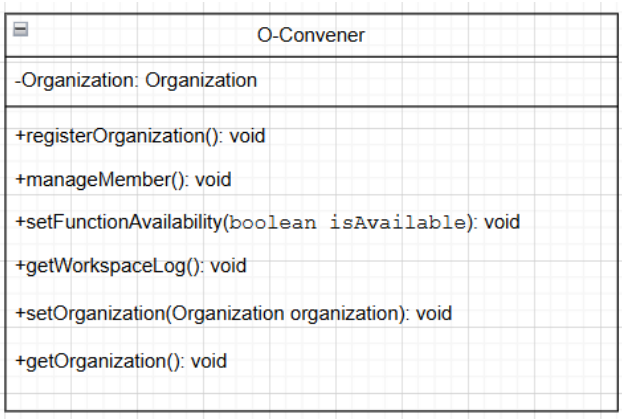


Figure 7 O-Convener

Explanation:

- **Organization**: Represents the core entity of an organization, including properties like name, ID, member list, and workspace configurations.
- **registerOrganization():** Registers a new organization in the system. Validates uniqueness of the organization name and assigns a unique ID.
- **manageMember()**: Allows administrators to add, remove, or modify members within the organization.
- **setFunctionAvailability(boolean isAvailable)**: Enables or disables specific functionalities for the organization (e.g., disabling member registration temporarily).
- **getWorkspaceLog()**: Retrieves logs of activities within the organization's workspace.
- **setOrganization(Organization organization)**: Updates the organization's metadata.
- **getOrganization():** Returns the current organization's details as an Organization object.

3.2.5 UserRegisManageSubsystem

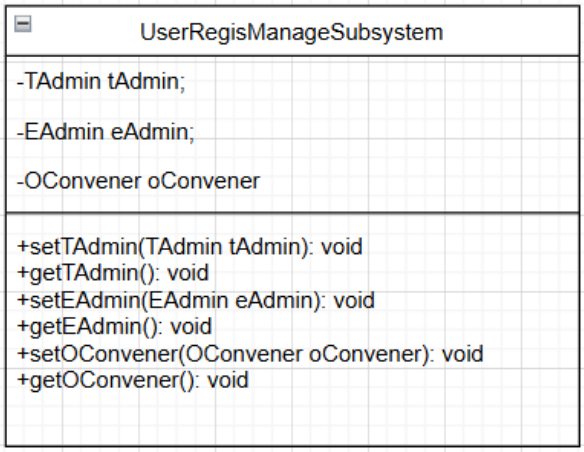


Figure 8 UserRegisManageSubsystem

Explanation:

- **tAdmin**: Represents a Technical Administrator (TAdmin) responsible for system-level configurations.
- **eAdmin**: Represents an Executive Administrator (EAdmin) handling user registration and policy management.
- **OConvener**: Represents an Organization Convener (OConvener) managing organization-level operations.
- **setTAdmin(TAdmin tAdmin)**: Assigns a technical administrator to the subsystem. Requires administrative privileges.
- **getTAdmin():** Returns the currently assigned TAdmin instance.
- **setEAdmin(EAdmin eAdmin):** Assigns an executive administrator to manage user registrations and policies.
- **getEAdmin()**: Returns the currently assigned EAdmin instance.
- **setOConvener(OConvener oConvener)**: Assigns an organization convener to oversee organization-specific operations.
- **getOConvener()***:* Returns the currently assigned OConvener instance.

3.2.6 ServiceConfiguration

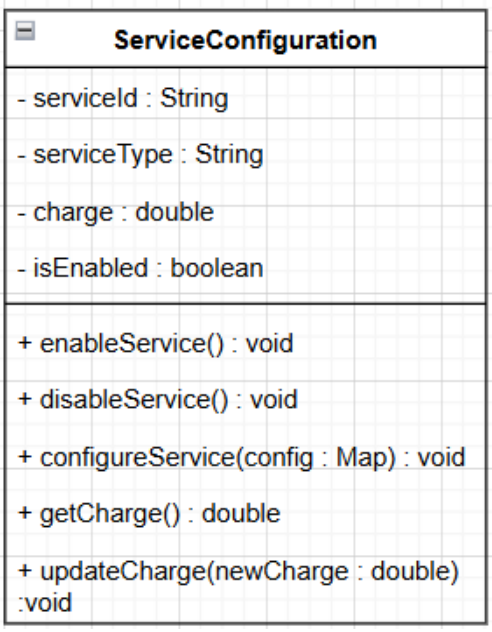


Figure 9 ServiceConfiguration

Explanations

- **serviceType**: Used to differentiate among various types of services (e.g., "CourseSharing", "ThesisSharing", etc.).
- **enableService()**: Sets isEnabled to true and executes any necessary initialization logic for enabling the service.
- **disableService()**: Sets isEnabled to false, stopping or disabling the related resources.
- **configureService(config : Map)**: Updates the internal settings of the service based on the provided configuration parameters (for example, database connection information).
- **getCharge()**: Returns the current charge value for the service.
- **updateCharge(newCharge : double)**: Updates the charge value to the new specified amount.

3.2.7 CourseConfig

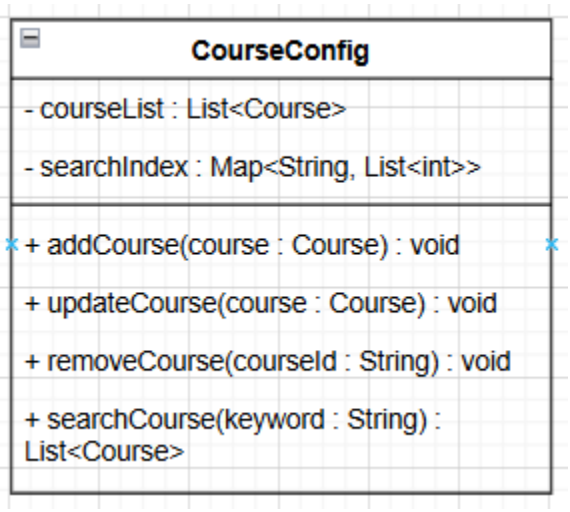


Figure 10 CourseConfig

Explanations

- **searchIndex**: A mapping structure maintained to speed up keyword searches; the key represents a keyword, and the value is a list of indices corresponding to specific course IDs.
- **addCourse(course : Course)**: Adds a new Course object to the courseList and updates the search index accordingly.
- **updateCourse(course : Course)**: Finds and replaces the existing course (matching by courseId) with updated course details.
- **removeCourse(courseId : String)**: Removes the course with the specified courseId from courseList and updates the search index.
- **searchCourse(keyword : String)**: Searches for courses that match the provided keyword (either using the searchIndex or by iterating over courseList) and returns the list of matching courses.

3.2.8 InterfaceConfig

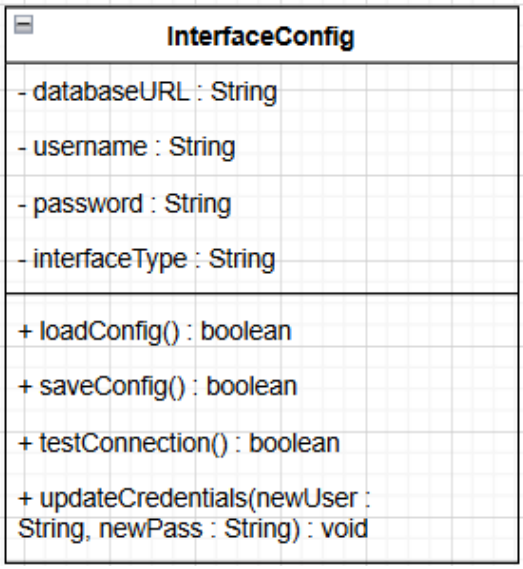


Figure 11 InterfaceConfig

Explanations

- **interfaceType**: Used to differentiate among different interface scenarios (such as "StudentAuthDB", "ThesisDB", "PaymentGateway", etc.).
- **loadConfig()**: Loads the current interface configuration (such as URL, username, and password) from persistent storage (a file, database, etc.) and assigns the values to the corresponding attributes.
- **saveConfig()**: Persists the current configuration (attributes such as URL, username, and password) to permanent storage.
- **testConnection()**: Attempts a connection using the current databaseURL, username, and password; returns true if the connection is successful, otherwise false.
- **updateCredentials(newUser : String, newPass : String)**: Updates the credentials with the new values provided; this operation may then require calling saveConfig() to persist these changes.

3.2.9 Payment

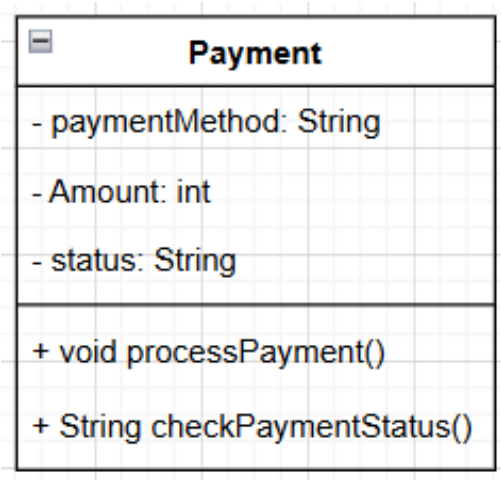


Figure 12 Payment

Explanations

- **processPayment()**: Initiates payment transaction.
- **checkPaymentStatus()**: Returns current payment status matching the status attribute.

3.2.10 DataUser

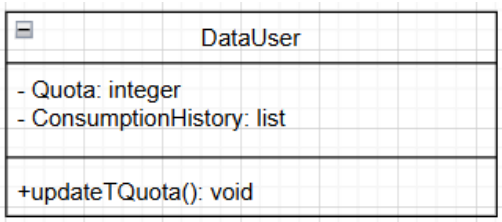


Figure 13 DataUser

Explanations

- **updateTQuota()**: Adjusts monthly thesis download limit.

3.2.11 PrivateDataProvider

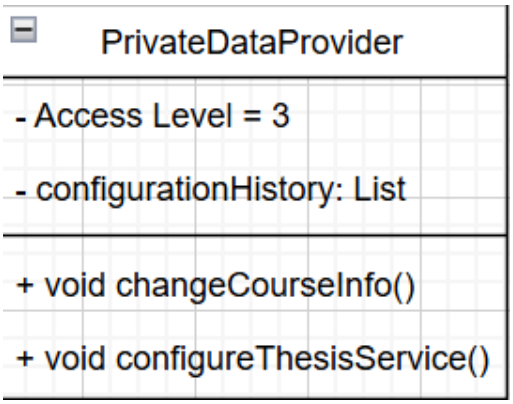


Figure 14 PrivateDataProvider

Explanations

- **changeCourseInfo()**: Modifies core course metadata.
- **configureThesisService()**: Adjusts thesis service parameters.

3.2.12 PrivateDataConsumer

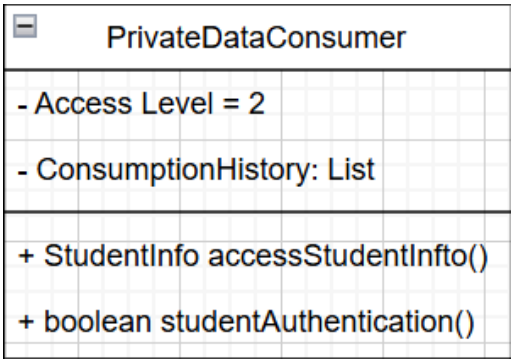


Figure 15 PrivateDataConsumer

Explanations

- **accessStudentInfo()**: Returns complete student profile.

- **studentAuthentication()**: Verifies student credentials.

3.2.13 PublicDataConsumer

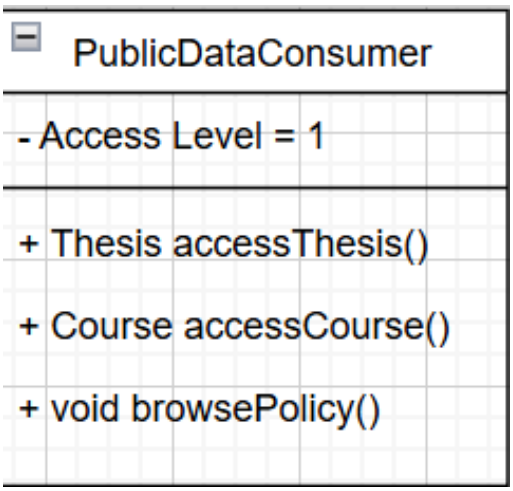


Figure 16 PublicDataConsumer

Explanations

- **accessThesis()**: Grants thesis viewing access without data modification.
- **accessCourse()**: Provides course material access.
- **browsePolicy()**: Displays institutional policies.

3.2.14 Service

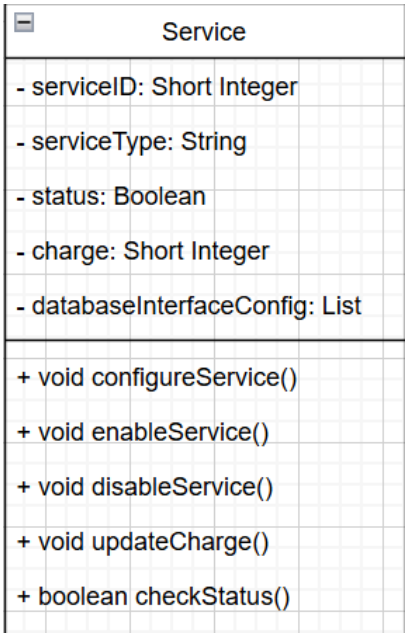


Figure 17 Service

Explanations

- **configureService()**: Sets up service parameters.
- **enableService()**: Activates service.
- **disableService()**: Deactivates service.
- **updateCharge()**: Modifies usage fees.
- **checkStatus()**: Returns service health status.

3.2.15 CourseService

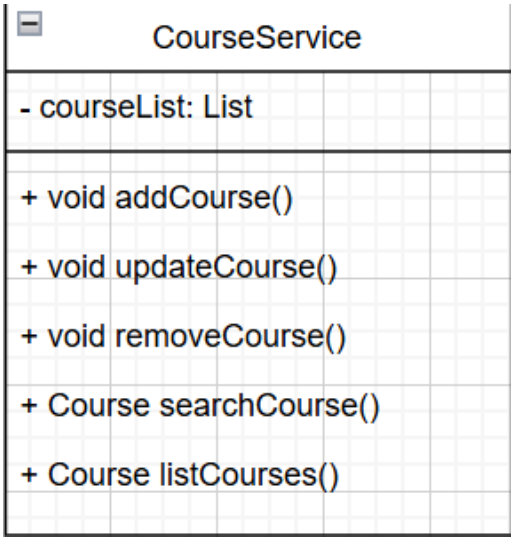


Figure 18 CourseService

Explanations

- **addCourse()**: Creates new course entry.
- **updateCourse()**: Modifies existing course details.
- **removeCourse()**: Deletes course from system.
- **searchCourse()**: Returns filtered course results.
- **listCourses()**: Returns all available courses.

3.2.16 ThesisService

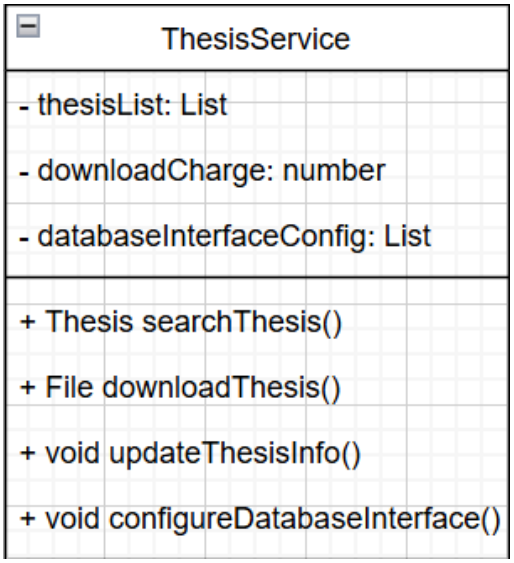


Figure 19 ThesisService

Explanations

- **searchThesis()**: Returns matching thesis documents.
- **downloadThesis()**: Returns thesis file content.
- **updateThesisInfo()**: Edits thesis metadata.
- **configureDatabaseInterface()**: Adjusts DB connection settings.

3.2.17 StudentIdentityService

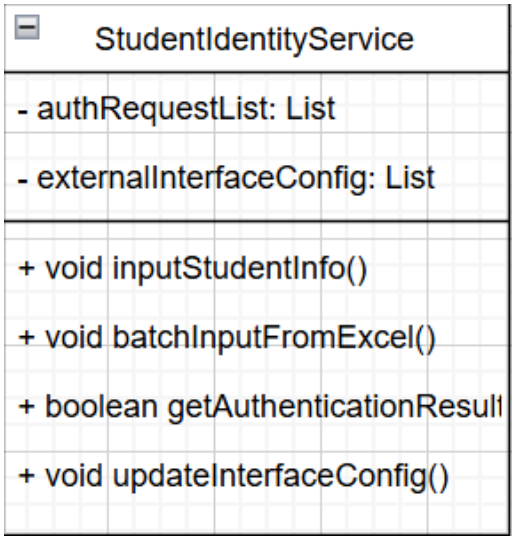


Figure 20 StudentIdentityService

Explanations

- **inputStudentInfo()**: Manual identity record creation.

- **batchInputFromExcel()**: Bulk identity data import.
- **getAuthenticationResult()**: Returns auth verification status.
- **updateInterfaceConfig()**: Modifies external system connections.

3.2.18 StudentRecordService

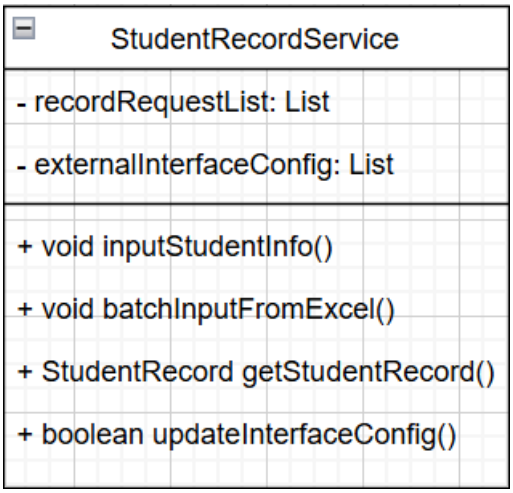


Figure 21 StudentRecordService

Explanations

- **inputStudentInfo()**: Manual student data entry.
- **batchInputFromExcel()**: Bulk import from spreadsheet.
- **getStudentRecord()**: Returns complete academic history.
- **updateInterfaceConfig()**: Modifies external system connections.

3.2.19 ActivityRecord (LOG)

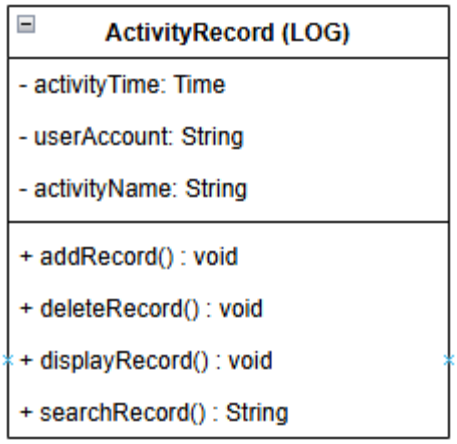


Figure 22 ActivityRecord

Explanation:

- **ActivityRecord:** Represents a log of user activities, capturing details about when (activityTime), which user (userAccount), and what activity (activityName) occurred. **addRecord():** Adds a new activity record to the log. Ensures timestamps are recorded accurately and user account references are valid.
- **deleteRecord():** Deletes an activity record, possibly after verification of authorization to modify logs.
- **displayRecord():** Displays activity records, formatted for readability (e.g., showing timestamps, user accounts, and activity names).
- **searchRecord():** Searches for activity records based on criteria (e.g., user account or activity name) and returns the result as a String.

3.2.20 UserQuestion

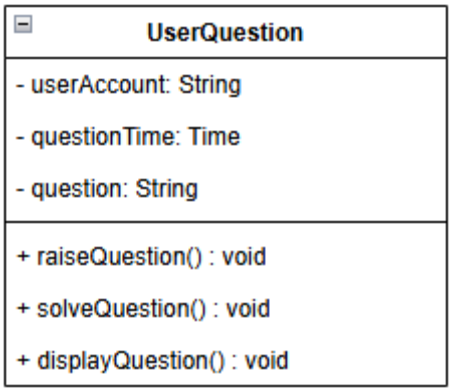


Figure 23 UserQuestion

Explanation:

- **UserQuestion**: Represents a question raised by a user, storing the user's account (userAccount), the time of the question (questionTime), and the question content (question).
- **raiseQuestion()**: Allows a user to submit a new question, validating that userAccount is logged - in and question content is not empty.
- **solveQuestion()**: Marks a question as solved, possibly updating its status and notifying the user.
- **displayQuestion()**: Shows the question details (user account, time, and content), formatted for display (e.g., in a user interface or report).

3.2.21 StudentRecordList

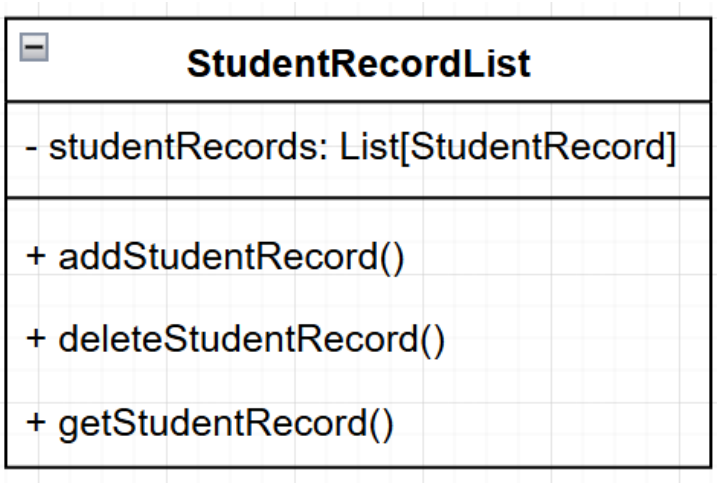


Figure 24 StudentRecordList

Explanations

- **addStudentRecord()**: add a new student record item
- **deleteStudentRecord()**: delete the student record item
- **getStudentRecord()**: get the student record in the list

3.2.22 StudentRecord

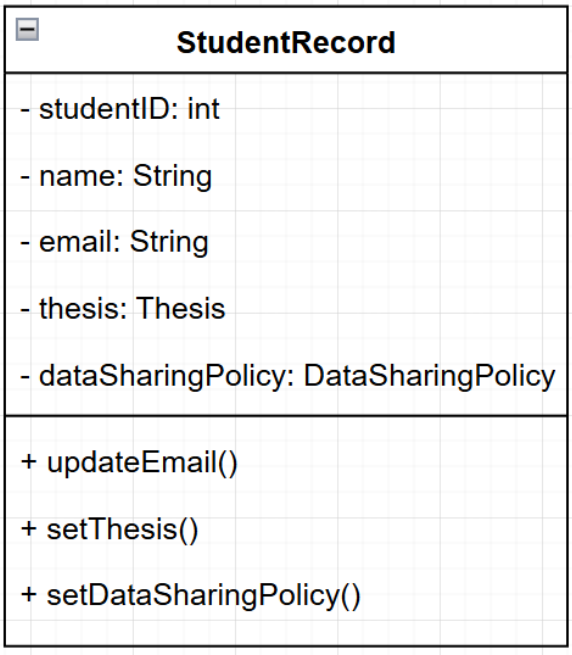


Figure 25 StudentRecord

Explanations

- **updateEmail()**: update student's email
- **setThesis()**: set or update thesis in the record
- **setDataSharingPolicy()**: set or update data sharing policy

3.2.23 Thesis

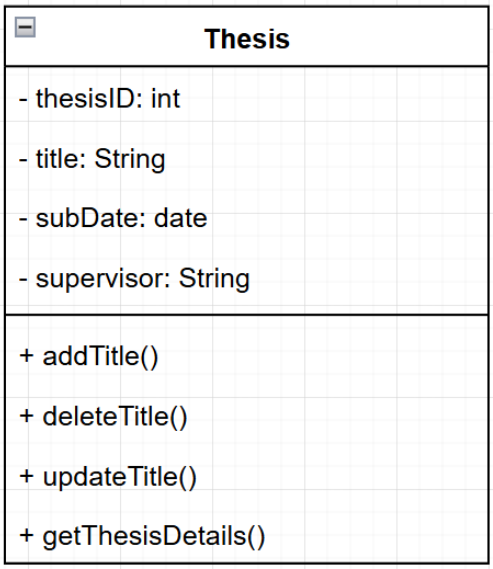


Figure 26 Thesis

Explanations

- **addTitle()**: add the title of the thesis
- **deleteTitle()**:delete the title of the thesis
- **updateTitle()**: update the title of the thesis
- **getThesisDetails()**: return the details of the thesis

3.2.24 Email

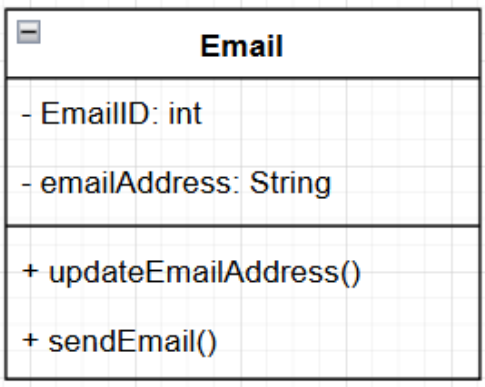


Figure 27 Email

Explanations

- **updateEmailAddress()**: update the student's email address.
- **sendEmail ()**: send an email to a student

Constraints

3.2.25 DataSharingPolicy

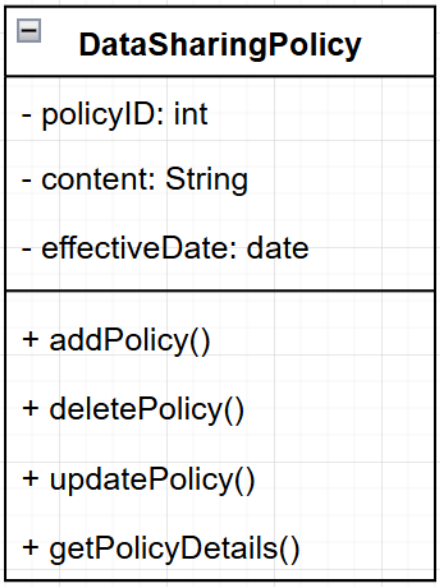


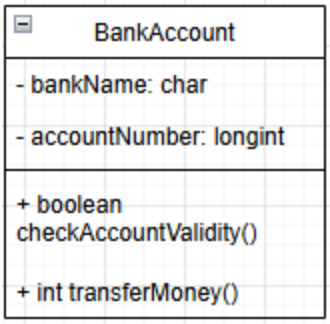
Figure 28 DataSharingPolicy

Explanations

- **addPolicy()**: add a new data sharing policy
- **deletePolicy()**: delete the data sharing policy
- **updatePolicy()**: update the content of data sharing policy

- **getPolicyDetails()**: return the details of the data sharing policy

3.2.26 BankAccount

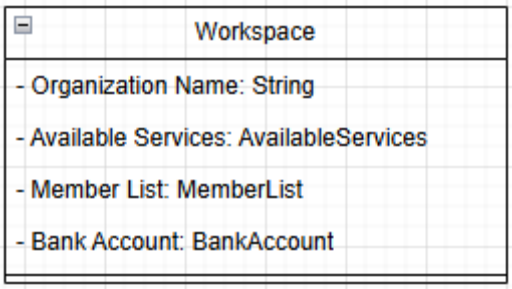


Explanations:

- **checkAccountValidity()**: Check if the given account number belongs to one of the created bank account**.** Return **true** if account is valid, otherwise **false**.

- **transferMoney()**: Transfer a given number of money to receiver's bank account. Return the number of successfully transferred money amount.

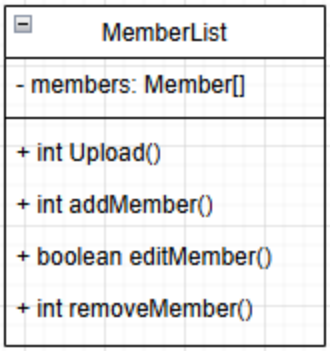
3.2.27 Workspace



Explanations:

Workspace doesn't need any methods for itself, because all functions are implemented within its attribute objects.

3.2.28 MemberList



Explanations:

- **Upload()**: Upload a spreadsheet file that contains information of member(s). Members will be automatically created with given information. Returns the count of created member(s).

- **addMember()**: Add a member with given information passed from arguments. Returns the count of created member, i.e. 1 if the member is successfully created, 0 if operation failed.

- **editMember()**: Edit an existing member's information with given information passed from arguments. Returns whether edit operation is successful.

- **removeMember()**: Remove members(s) passed from argument. Returns the count of removed member(s).

4. Alternative detail design

Null

5. More considerations

To further enhance the system's stability, reusability, and scalability, the following recommendations should be considered:
Stability Improvements:
Define Clear Interfaces: Ensure all inter-module communications (e.g., between Payments and Service) follow strict contracts (APIs or events) to prevent unintended dependencies.
Reusability Enhancements:
Standard Shared Components: Create a Common Utilities module for reusable functions (e.g., Email, Upload) to avoid duplication.
Scalability Strategies:
Adopt Event-Driven Architecture: For future expansions (e.g., notifications), use messaging queues to avoid tight coupling between modules.